

A Proof of Proposition 1

As shown in Algorithm 2, the first step (Line 1) constructs distance matrices for all attributes by calling the algorithm PDM. For each attribute A_j , PDM divides the rows of the distance matrix among the $|\mathcal{P}|$ processors so that each processor computes the distances for all tuple pairs in its assigned rows. Since there are at most m attributes and $n \times n$ tuple pairs, the overall cost is $O\left(\frac{m \cdot n^2}{|\mathcal{P}|}\right)$.

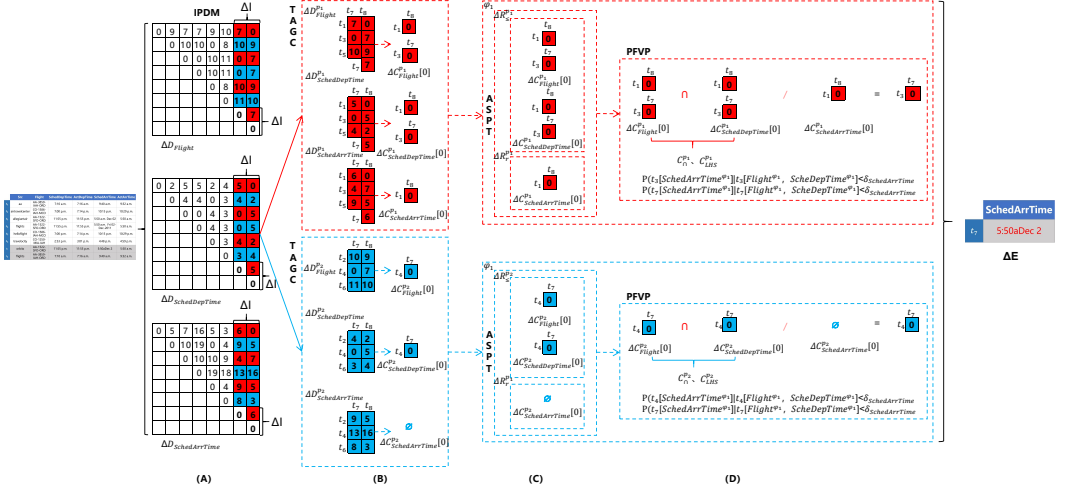


Fig. 11. An Overview of Our Incremental Parallel Detection (IPED)

Next, Algorithm 2 invokes TAGC (Line 3) to generate clusters for each attribute, as presented in Algorithm 1. In particular, Line 1 sorts the threshold set Θ_j in constant time, because the size of Θ_j is small, while Line 2 must traverse the entire distance matrix in $O(n^2)$ time to obtain $C_j[\theta_j^{\max}]$. Then, in Lines 3–5, TAGC refines the maximum-threshold cluster without a second full traversal, but instead iterates over the cluster computed in the previous step. Throughout this process, the distance matrix rows are partitioned among the $|\mathcal{P}|$ processors, so that each processor handles clustering for all relevant thresholds in Θ_j within its assigned rows. Consequently, generating clusters for every attribute in parallel over at most m attributes requires $O\left(\frac{m \cdot n^2}{|\mathcal{P}|}\right)$ time overall.

Afterward, in Lines 4–17, Algorithm 2 conducts the complete error detection process for every constraint $\phi \in \Sigma$. Specifically, in Line 6, the algorithm ASPT partitions the tuple pair sets from the attribute clusters of ϕ across $|\mathcal{P}|$ processors in parallel. Since Lines 2-3 generate clusters in parallel for up to m attributes, each processor already holds the tuple pair subsets of the attributes involved in ϕ . Therefore, ASPT can directly use these local subsets to construct R_C^p , R_S^p , and R_T^p in parallel. As a result, this step completes in $O(m)$ time. Next, in Line 7, Algorithm 2 calls PFVP to find violating tuple pairs by performing set operations (intersection and difference) over the tuple pair sets derived from the attribute clusters of ϕ , each of which contains at most n^2 tuple pairs. The row-based partitioning strategy also applies here: each processor works on the tuple pair sets of rows assigned to it, carrying out intersection and difference in parallel. As a result, executing these set operations for at most m attributes among $|\mathcal{P}|$ processors costs $O\left(\frac{m \cdot n^2}{|\mathcal{P}|}\right)$. Having obtained all violating tuple pairs C_ϕ^p of ϕ , Lines 8–17 traverse each tuple pair (t_i, t_j) and examine up to $2 \times m$ cells per pair to identify erroneous cells in parallel, thus taking $O\left(\frac{m \cdot n^2}{|\mathcal{P}|}\right)$ time in that final step.

Summing up, for each constraint φ in Σ , the cost is $O\left(\frac{m \cdot n^2}{|\mathcal{P}|}\right)$. Because there are $|\Sigma|$ such constraints, the total time complexity of PED is $O\left(\frac{|\Sigma| \cdot m \cdot n^2}{|\mathcal{P}|}\right)$ as stated in proposition 1.

B Proof of Proposition 2

Algorithm IPED begins by calling IPDM, which incrementally updates the distance matrices from size $n \times n$ to $(n + \Delta n) \times (n + \Delta n)$. Since only rows $1 \dots (n + \Delta n)$ and columns $(n + 1) \dots (n + \Delta n)$ must be filled, each of the $|\mathcal{P}|$ processors is assigned a subset of these rows and computes the tuple pair distances within them, resulting in a total cost of $O\left(\frac{m \cdot (n + \Delta n) \cdot \Delta n}{|\mathcal{P}|}\right)$ for at most m attributes.

Next, the algorithm IPED invokes TAGC to build incremental clusters ΔC_j for each attribute A_j based on the incremental distance matrix ΔD_j . TAGC partitions the rows $1 \dots (n + \Delta n)$ among the $|\mathcal{P}|$ processors, generating the clusters for its assigned rows. Since the number of relevant tuple pairs is reduced from $(n + \Delta n)^2$ to $(n + \Delta n) \cdot \Delta n$, the cost of generating clusters for at most m attributes is $O\left(\frac{m \cdot (n + \Delta n) \cdot \Delta n}{|\mathcal{P}|}\right)$.

Afterward, for each constraint $\varphi \in \Sigma$, the algorithm IPED detects errors incrementally. Since the attribute clusters have already been generated, and the tuple pair subsets for the attributes involved in φ are already available on each processor during the TAGC phase, ASPT can construct $R_{\leq}^p$, $R_{>}^p$, and R_r^p in parallel based on local results with $O(m)$ cost per processor. Then, PFVP finds violating tuple pairs by set operations (intersection and difference) over these tuple pair sets. Each tuple pair set contains up to $(n + \Delta n) \cdot \Delta n$ tuple pairs, so carrying out these operations for at most m attributes in parallel takes $O\left(\frac{m \cdot (n + \Delta n) \cdot \Delta n}{|\mathcal{P}|}\right)$. Once PFVP identifies the violating tuple pairs ΔC_{φ}^p , the algorithm IPED scans each tuple pair (t_i, t_j) and examines up to $2 \times m$ cells per pair to detect erroneous cells, taking $O\left(\frac{m \cdot (n + \Delta n) \cdot \Delta n}{|\mathcal{P}|}\right)$ time for that final step.

Hence, each constraint $\varphi \in \Sigma$ requires $O\left(\frac{m \cdot (n + \Delta n) \cdot \Delta n}{|\mathcal{P}|}\right)$ time for incremental detection, and thus the total complexity over all $|\Sigma|$ constraints is $O\left(\frac{|\Sigma| \cdot m \cdot (n + \Delta n) \cdot \Delta n}{|\mathcal{P}|}\right)$ as stated in proposition 2.

Received January 2025; revised April 2025; accepted May 2025